

PARADIGMAS DE PROGRAMACIÓN: PROYECTO 1. MÁQUINA DE TURING.

Andrés Sebastián Coral Vallejo & Juan Pablo Orjuela.

1. LA MÁQUINA DE TURING.

La máquina de Turing es un modelo matemático de computación propuesto por el matemático Alan Turing en 1936, concebido como una herramienta teórica para estudiar los límites del cálculo y la resolución de problemas mediante procedimientos mecánicos. Se considera uno de los pilares fundamentales de la informática teórica, ya que formaliza la noción de algoritmo y define qué es computable en un sentido estricto.

La máquina de Turing está compuesta por múltiples componentes como los visibles en la *Tabla 1*.

Cinta infinita.	Es una secuencia de celdas dispuestas linealmente, que puede extenderse indefinidamente en ambas direcciones. Cada celda puede contener un símbolo perteneciente a un alfabeto finito. La cinta actúa simultáneamente como memoria y como medio de entrada/salida.
Cabezal de lectura y escritura.	Se desplaza a lo largo de la cinta, leyendo el símbolo en la celda actual, escribiendo uno nuevo si es necesario y moviéndose a la izquierda o a la derecha según lo indique la función de transición.
Conjunto finito de estados.	La máquina dispone de un número limitado de estados internos, incluido uno inicial y al menos uno de aceptación o rechazo. El estado actual, junto con el símbolo leído en la cinta, determina la acción siguiente.
función de transición.	Es la regla formal que indica qué hacer en cada situación: - Qué símbolo escribir en la cinta. - Hacia qué dirección mover el cabezal (izquierda o derecha). - Hacia qué estado transitar.

Tabla 1. Componentes de la máquina de Turing.

Aunque es común asociar la máquina de Turing con un alfabeto binario (0 y 1), no está limitada exclusivamente a dicho sistema. El alfabeto puede estar compuesto por cualquier conjunto finito de símbolos, por ejemplo {0, 1, □} (donde □ representa un espacio en blanco), o incluso letras y otros caracteres.

Como fue mencionado previamente, la máquina de Turing es un modelo matemático, sin embargo, este se puede representar como una composición de los componentes mencionados, de forma física, no necesariamente abstracta, como se observa en la *Figura 1*. donde se muestra una ilustración esquemática de la maquina como si fuera un objeto físico que implementa esta lógica abstracta donde hay cintas infinitas en las que la maquina puede leer o escribir.

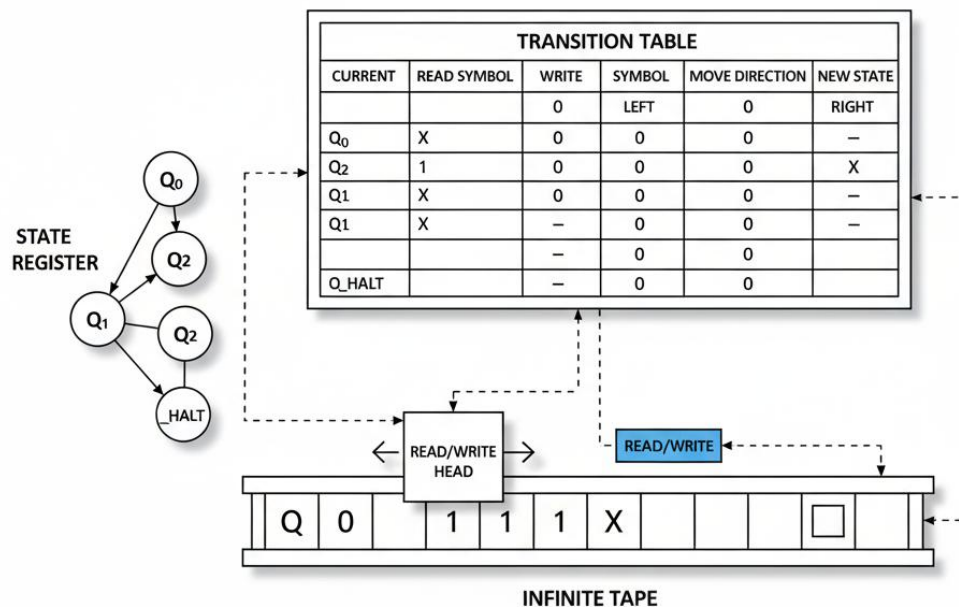


Figura 1. Ilustración esquemática de la máquina de Turing – Gemini AI.

En un sentido físico, la idea de una cinta infinita es imposible, pero en el ámbito computacional esta puede ser simulada de diferentes formas como:

- Arreglos o listas dinámicas:

Se utilizan estructuras de datos que pueden expandirse según sea necesario (como listas enlazadas o arreglos dinámicos), lo que permite “simular” el crecimiento indefinido de la cinta en la medida en que la máquina lo requiera.

- Memoria virtual y direccionamiento:

En entornos más formales, la cinta se implementa como una secuencia de posiciones de memoria, con la capacidad de extenderse hacia la izquierda o hacia la derecha mediante reasignación dinámica de memoria.

En ese orden de ideas y aunque teóricamente la cinta no tiene fin, en la práctica se establece un límite máximo de memoria que, en la mayoría de los casos experimentales o académicos, nunca llega a ser un impedimento, ya que los problemas se diseñan con un número finito y razonable de pasos.

2. CONCEPTUALIZACIÓN DEL DISEÑO.

Para el proyecto se solicitó una máquina de Turing con 6 operaciones fundamentales (Suma, resta, multiplicación, división, potenciación, raíz cuadrada) implementando un método para cada una. Para esto fue necesario el llegar a comprender la lógica detrás de la maquina y poder definir los pasos que sigue para llegar a un resultado en concreto.

Como se mencionó previamente, la máquina de Turing no está limitada exclusivamente al sistema binario, por lo que para concretar el diseño era vital elegir entre un diseño con un sistema binario en mente o un sistema distinto. Finalmente se decidió por desarrollar la maquina considerando el sistema binario para facilitar algunas operaciones como la potencia y la raíz. Así mismo se propuso simular una MT (Maquina de Turing) multi-cinta que constaría de 3 cintas:

- Cinta A: entrada (números y separadores)
- Cinta B: trabajo/auxiliar (llevadas, desplazamientos, resto)
- Cinta C: resultado / salida

Así, se desarrolló un diagrama de flujo para visualizar la lógica programática de la máquina. Desde su inicio hasta la muestra de resultados y la salida, pasando por las diferentes estancias que el programa debe recorrer para llegar al resultado esperado, esta representación gráfica se puede apreciar en la *Figura 2.* que muestra de forma grafica el proceso que ejecuta la maquina y como busca llegar a sus resultados.

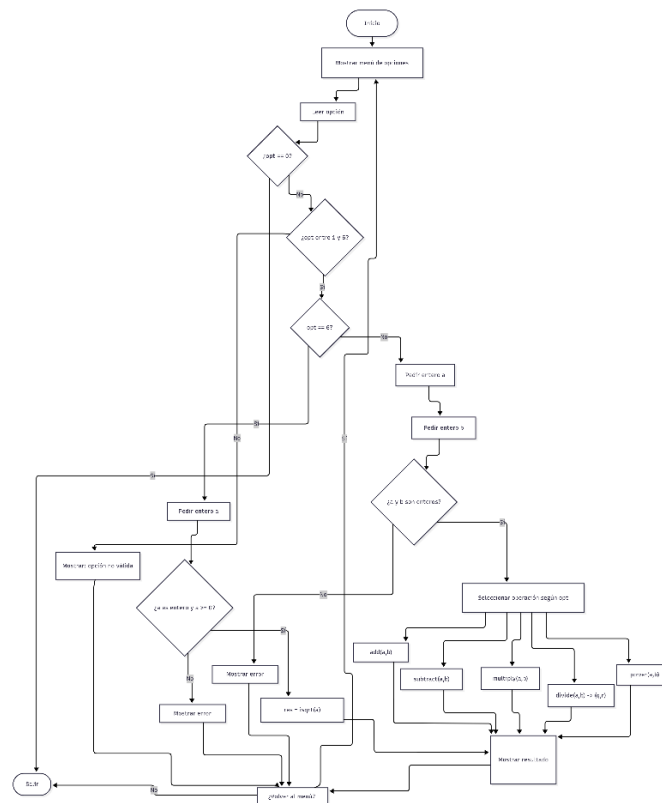


Figura 2. Diagrama de flujo de la máquina de Turing.
(Versión expandida al final del documento)



Posteriormente y con la conceptualización de los procesos de la maquina ya establecidos, se empezó a desarrollar el pseudocódigo respectivo para cada operación, con la intencionalidad de comprender e interiorizar los procesos que se ejecutan y como lo hacen logrando así un proceso de codificación más organizado y efectivo. Los pseudocódigos respectivos se pueden observar desde la *Tabla 2.* hasta la *Tabla 11.* Y en ese orden de ideas, es necesario aclarar unas cosas del diseño:

- Para la representación se usó (signo, binario) con binario MSB-left y sin ceros lideres para que los algoritmos bit a bit sean más claros.
- Cada operación se mapea a acciones sobre la cinta (leer bit, escribir bit, desplazar cabeza), por ejemplo: La suma itera LSB→MSB con carry, la multiplicación hace desplazamientos y sumas parciales, la división mantiene un resto parcial que equivale a mover y comparar segmentos de cinta, etc.
- Se consideraron algunas limitaciones como que solo se aceptan enteros, no se admiten exponentes negativos, la raíz cuadrada es solo válida para entradas mayores o iguales a 0 o que la división por cero tampoco está admitida.

Así mismo, para el planteamiento se establecieron algunos objetivos con la intención de hacer una simulación más fiel al concepto de la máquina de Turing, como lo fueron:

- **Objetivo 1:** Convertir entre enteros y representación (signo + binario sin ceros líderes), comparar binarios y limpiar ceros.
- **Objetivo 2:** Sumar dos enteros a y b respetando signos, usando operaciones sobre sus partes binarias e implementar la resta con el mismo proceso, pero con el signo invertido.
- **Objetivo 3:** Implementar suma y resta bit a bit desde el LSB (derecha) usando carry para la suma y borrow para la resta; devuelven cadenas binarias sin ceros líderes.
- **Objetivo 4:** Multiplicar respetando signos, usando desplazamientos y sumas binarias.
- **Objetivo 5:** Calcular (cociente, resto) con división entera truncada hacia cero y resto con signo del dividendo; manejar división por cero.
- **Objetivo 6:** Lograr que la implementación de la división use `division_larga_binaria` para no trabajar con conversiones enteras directas, simulando manipulación de cintas.
- **Objetivo 7:** Elevar $base^{exp}$ con exp entero no negativo, manteniendo el signo correcto.
- **Objetivo 8:** Conseguir la raíz cuadrada al devolver `floor(sqrt(n))` para $n \geq 0$ usando búsqueda binaria y multiplicación binaria para comparar y hacerlo usando multiplicación binaria `bin_mul` para mantener la filosofía de manipular bits/cintas.

Así y con ese planteamiento y objetivos claros, se desarrollaron los pseudocódigos respectivos para las operaciones.



FUNCIÓN int_a_signo_bin(n)
<pre>// Convierte entero n en (signo, binario_sin_signo) SI n == 0 ENTONCES retornar ('+', '0') FIN_SI SI n < 0 ENTONCES signo = '-' SINO signo = '+' FIN_SI binario = representación_binaria_de(abs(n)) // por ejemplo 6 será '110' binario = quitar_ceros_izquierda(binario) retornar (signo, binario)</pre>

Tabla 2. Pseudocódigo de la función que convierte enteros a signo, binario.

FUNCIÓN signo_bin_a_int(signo, binario)
<pre>valor = entero_desde_binario(binario) // por ejemplo '110' será 6 SI valor == 0 ENTONCES retornar 0 FIN_SI SI signo == '-' ENTONCES retornar -valor SINO retornar valor FIN_SI</pre>

Tabla 3. Pseudocódigo de la función que convierte binario a enteros.



FUNCIÓN quitar_ceros_izquierda(binario)
Eliminar todos los ceros al inicio de la cadena. SI la cadena queda vacía ENTONCES retornar '0' SINO retornar cadena limpia FIN_SI

Tabla 4. Pseudocódigo de la función que elimina los ceros al inicio de la cadena.

FUNCIÓN comparar_binarios(a, b)
// Compara dos cadenas binarias sin signo (MSB a la izquierda) limpiar ambos con quitar_ceros_izquierda. SI longitud(a) > longitud(b) ENTONCES retornar 1 SI longitud(a) < longitud(b) ENTONCES retornar -1 SI a > b (lexicográficamente) ENTONCES retornar 1 SI a < b ENTONCES retornar -1 retornar 0 // En caso de que a y b sean iguales

Tabla 5. Pseudocódigo de la función que compara los binarios.



FUNCIÓN suma(a, b):

```
(signoA, binA) = int_a_signo_bin(a)
(signoB, binB) = int_a_signo_bin(b)

SI binA == '0' Y binB == '0' ENTONCES
    retornar 0
FIN_SI

SI signoA == signoB ENTONCES // Misma dirección: sumar las magnitudes y conservar signo
    suma_abs = suma_binaria(binA, binB)
    retornar signo_bin_a_int(signoA, suma_abs)

SINO // Signos distintos: restar la menor magnitud de la mayor
    cmp = comparar_binarios(binA, binB)

    SI cmp == 0 ENTONCES
        retornar 0
    FIN_SI

    SI cmp > 0 ENTONCES // |a| > |b|
        diff_abs = resta_binaria(binA, binB) // a - b en valores absolutos
        retornar signo_bin_a_int(signoA, diff_abs) // signo del mayor (a)

    SINO // |b| > |a|
        diff_abs = resta_binaria(binB, binA)
        retornar signo_bin_a_int(signoB, diff_abs) // signo del mayor (b)
    FIN_SI

FIN_SI
```

Tabla 6. Pseudocódigo de la función que ejecuta la suma.



FUNCIÓN resta(a, b)
<pre>// Se utiliza el mismo método de cálculo que para la suma. // En este sentido se puede interpretar como un simple: a - b = a + (-b) retornar suma(a, -b)</pre>

Tabla 7. Pseudocódigo de la función que ejecuta la resta.

FUNCIÓN multiplicacion(a, b)
<pre>(signoA, binA) = int_a_signo_bin(a) (signoB, binB) = int_a_signo_bin(b) SI binA == '0' O binB == '0' ENTONCES retornar 0 resultado_abs = '0' // recorrer bits del multiplicador desde LSB hasta MSB PARA i = 0 HASTA longitud(binB)-1: bit = bit_de_binario(binB, desde_derecha=i) // i=0 -> LSB SI bit == '1' ENTONCES // agregar binA desplazado i posiciones a la izquierda agregado = binA + ('0' repetido i) // equivalente a shift left resultado_abs = suma_binaria(resultado_abs, agregado) FIN_SI FIN_PARA signo_resultado = '-' si signoA != signoB SINO '+' retornar signo_bin_a_int(signo_resultado, resultado_abs)</pre>

Tabla 8. Pseudocódigo de la función que ejecuta la multiplicación.



FUNCIÓN dividir(a, b)

```
SI b == 0 ENTONCES
lanzar_error("División por cero")

(signoA, binA) = int_a_signo_bin(a)
(signoB, binB) = int_a_signo_bin(b)

// División absoluta por algoritmo de división larga en base 2:
(cociente_abs, resto_abs) = division_larga_binaria(binA, binB)

// division_larga_binaria:
recorrer bits de binA de MSB a LSB
mantener un 'resto_parcial' (inicialmente '0')
en cada paso: resto_parcial = (resto_parcial << 1) + siguiente_bit

SI resto_parcial >= binB ENTONCES
    resto_parcial -= binB y escribir '1' en cociente
SINO
    escribir '0'

retornar (cociente, resto) // Retornar sin ceros adelante

// Determinar signo del cociente por XOR de signos
signo_cociente = '-' SI (a < 0) XOR (b < 0) SINO '+'
cociente = signo_bin_a_int(signo_cociente, cociente_abs)

// Calcular resto consistente con la definición: resto = a - (cociente * b)
resto = a - (cociente * b) // Esto preserva el signo del dividendo según convención usada
retornar (cociente, resto)
```

Tabla 9. Pseudocódigo de la función que ejecuta la división.



FUNCIÓN potencia(base, exp)

```
SI exp < 0 ENTONCES
    lanzar_error("Exponentes negativos no admitidos")

SI exp == 0 ENTONCES
    retornar 1 // por convención: cualquier base^0 = 1
    (signoBase, binBase) = int_a_signo_bin(base)

SI binBase == '0' ENTONCES
    retornar 0 // 0^exp = 0 para exp > 0

resultado_abs = '1' // almacenado en binario
base_actual = binBase
e = exp

MIENTRAS e > 0:

    SI (e mod 2) == 1 ENTONCES
        resultado_abs = bin_mul(resultado_abs, base_actual) // multiplicar en binario
    FIN_SI

    base_actual = bin_mul(base_actual, base_actual) // elevar al cuadrado
    e = floor(e / 2)

FIN_MIENTRAS

// signo final: negativo si base negativa y exp impar
signo_resultado = '-' si (signoBase == '-' Y (exp mod 2) == 1) SINO '+'
retornar signo_bin_a_int(signo_resultado, quitar_ceros_izquierda(resultado_abs))
```

Tabla 10. Pseudocódigo de la función que ejecuta la potenciación.



FUNCIÓN raiz_cuadrada_entera(n)

```
SI n < 0 ENTONCES
    lanzar_error("Raíz de número negativo no soportada")
    bajo = 0
    alto = n
    respuesta = 0

MIENTRAS bajo <= alto:
    medio = floor((bajo + alto) / 2)
    (signoM, binM) = int_a_signo_bin(medio)
    medio_cuadrado_bin = bin_mul(binM, binM)
    medio_cuadrado = entero_desde_binario(quitar_ceros_izquierda(medio_cuadrado_bin))

SI medio_cuadrado <= n ENTONCES
    respuesta = medio
    bajo = medio + 1

SINO
    alto = medio - 1

FIN_SI

FIN_MIENTRAS

retornar respuesta
```

Tabla 11. Pseudocódigo de la función que ejecuta la raíz cuadrada.

3. IMPLEMENTACIÓN.

Una vez desarrollado todo el proceso de conceptualización y diseño, se desarrolló la codificación y para lograrlo se consideraron las 10 funciones propuestas, así como los 8 objetivos propuestos para el desarrollo. En ese orden de ideas, se codificaron las funciones una a una en el proyecto de Python.

El proceso de codificación cubrió todo lo que se solicitó y lo que se propuso en el diseño de la máquina de Turing:

- 6 operaciones fundamentales (Suma, resta, multiplicación, división, potenciación, raíz cuadrada) como se solicitó originalmente.
- 16 funciones en total (4 para el manejo del sistema binario, 10 para las operaciones matemáticas y 2 para la interfaz de usuario en consola)
- Se cumplieron los 8 objetivos propuestos durante la conceptualización del diseño.

La implementación puede observarse en el archivo [Main.py](#) que se entregó junto a este informe y se encuentra estructurado de forma que sea comprensible que rol cumple cada función en el programa. En ese orden de ideas, las funciones se ven seccionadas por un comentario que denota su utilización general, de la siguiente forma:

- UTILITARIOS: conversiones y útiles para binarios con signo (4 funciones)
- OPERACIONES BINARIAS (manipulación de cadenas) — MSB a la izquierda (4 funciones)
- CLASE: TMSimulator — operaciones de alto nivel usando las funciones binarias (6 funciones)
- INTERFAZ: Menú y control de flujo (2 funciones)

E internamente cada función tiene sus propios comentarios que explican al usuario lo que se realiza en cierta parte del código, de esa forma se mantuvo el orden y la estructuración cómoda del código a la hora de escribirlo un ejemplo de la estructuración del código puede verse en la *figura 3*. Donde se observa el seccionamiento y adicionalmente los comentarios individuales y explicativos.

```
22 # -----
23 # UTILITARIOS: conversiones y utils para binarios con signo
24 # -----
25 def int_to_signed_bin(n: int) -> Tuple[str, str]:
26     """
27     Convierte un entero n a (signo, binstr).
28     - signo: '+' o '-'
29     - binstr: cadena binaria absoluta sin ceros iniciales, '0' si n == 0
30     Ejemplo: -6 -> ('-', '110')
31     """
32     if n == 0:
33         return '+', '0'
34     sign = '-' if n < 0 else '+'
35     b = bin(abs(n))[2:] # '0b...' -> cortamos '0b'
36     return sign, b
37
```

Figura 3. Ejemplo de la escritura del código.



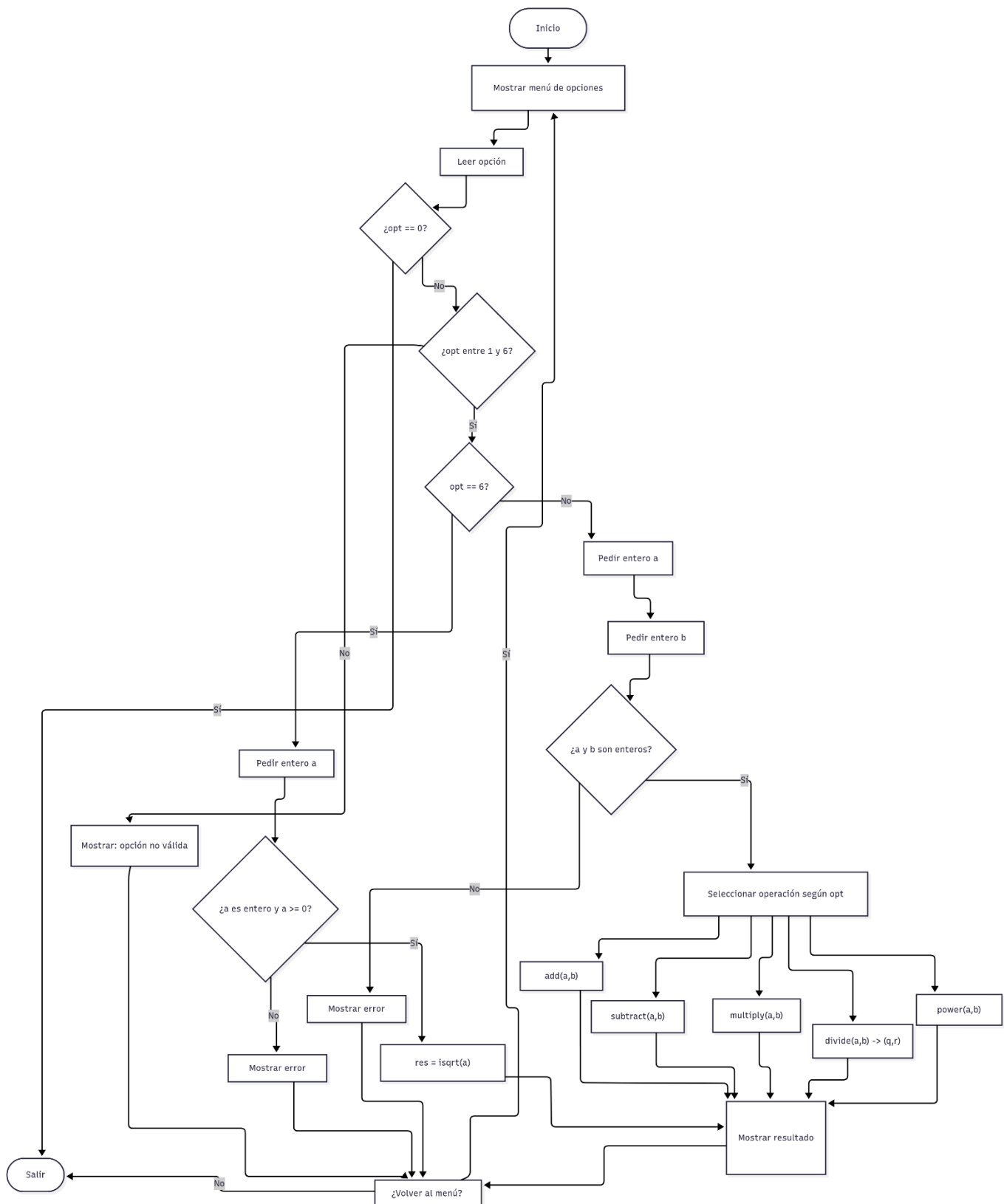
4. Casos de prueba.

Una vez realizado el código, se establecieron unos casos de prueba con el fin de comprobar la utilidad de la implementación, los casos de prueba se establecieron en el código, pero también se pueden ingresar de forma normal en el programa y no debería de producirse problema alguno, los casos de prueba pueden ser vistos en la Tabla 12. que muestra la estructura (Operación, entero a, entero b, y el resultado esperado después del cálculo).

OPERACIÓN	Entero a	Entero b	Resultado esperado
Suma	3	5	8
Suma	-3	5	2
Resta	10	4	6
Resta	4	10	-6
Multiplicación	7	6	42
Multiplicación	-7	6	-42
División	20	4	(5, 0)
División	7	3	(2, 1)
División	-7	3	(-2, -1)
Potenciación	2	3	8
Potenciación	-2	3	-8
Raíz cuadrada	16	Nada	4
Raíz cuadrada	15	Nada	3

Tabla 12. Casos de prueba.

Para ejecutar los casos de prueba solo es necesario copiar los elementos del archivo [CasosPrueba.txt](#) reemplazando la ultima parte del código, correspondiente a la sección PUNTO DE ENTRADA y los casos de prueba deberían ejecutarse antes de abrir el menú de opciones.



**Figura 2. Diagrama de flujo de la máquina de Turing.
(Versión expandida)**